



1 8 0 3

# INFORMÁTICA II

*Bioingeniería*





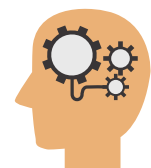
## Publicar en Github desde VS



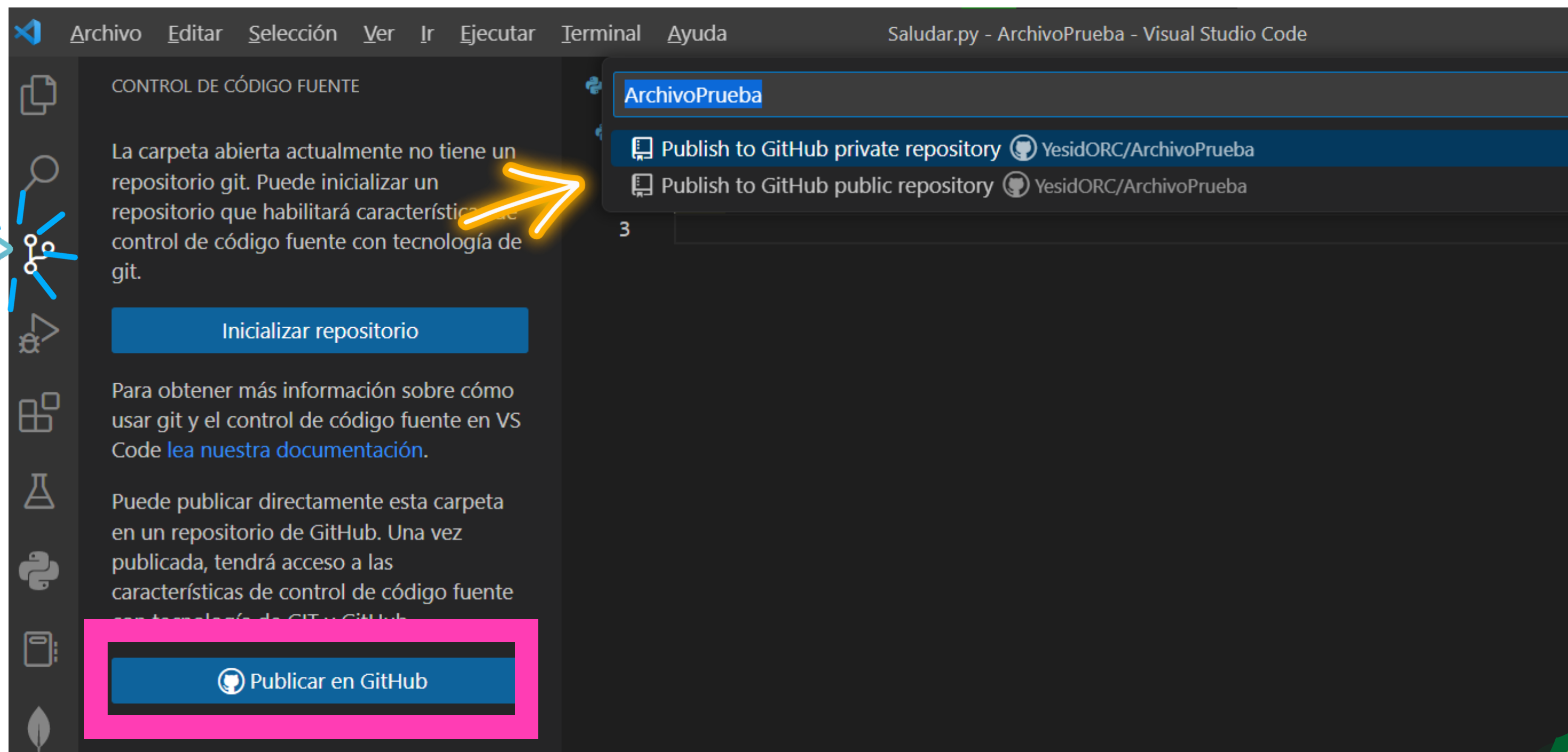
Para publicar un repositorio desde nuestra maquina local:

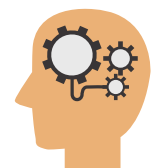
- Crear una carpeta y abrirla con VS
- Tener almenos un archivo allí (opcional para ser mas prácticos)
- Dirigirse a "control source" y darle en publish repository.
- Elegir, si lo desea crear privado o público
- Solicita agregar archivos , por defecto el que se creo inicialmente en la carpeta
- Y click en aceptar

Una vez realizado este proceso podemos continuar como hemos practicado en clase ,  
haceindo commits y sincronizando(push) cambios con GitHub

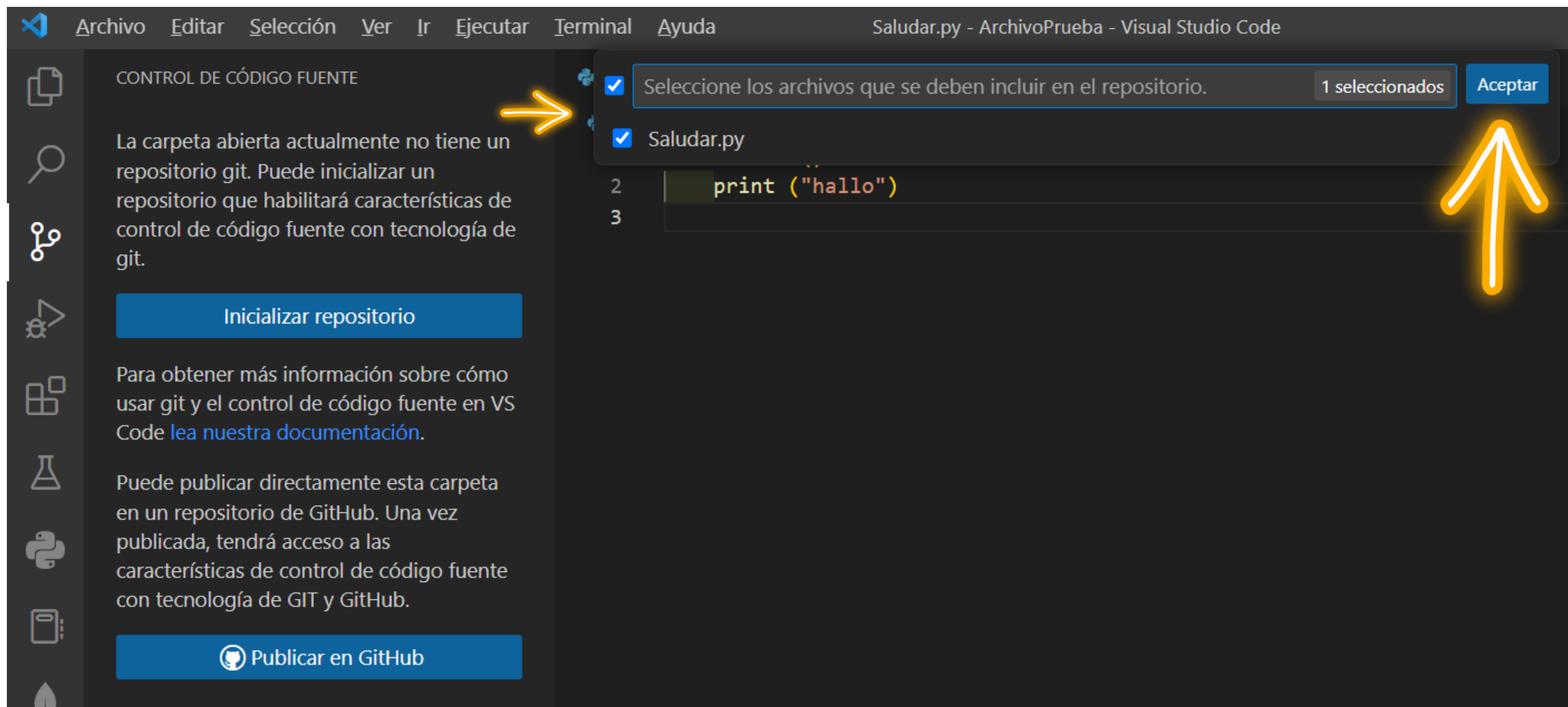


# Publicar en Github desde VS





# Publicar en Github desde VS





# Github





YesidORC

Edit profile

10 followers • 0 following

Overview Repositories 5 Projects Packages Stars

Find a repository... Type Language Sort New

ArchivoPrueba Public

Python Updated 4 minutes ago

hyperblogFork Public

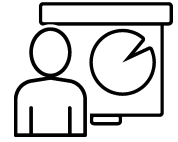
Forked from freddier/hyperblog

Un blog increíble para el curso de Git y Github de Platzi

HTML 11,919 Updated 2 days ago

info14-16 Public

Python 5 Updated 2 days ago



## Metodo Main



En programas con múltiples funciones es necesario indicar la función principal, main, que será la encargada de llamar las otras funciones

def main():

    #instrucciones programa

    #indicacion a python para que comience la ejecución en método

    #main

```
88 | ✓ if __name__ == '__main__':  
89 |     main()
```



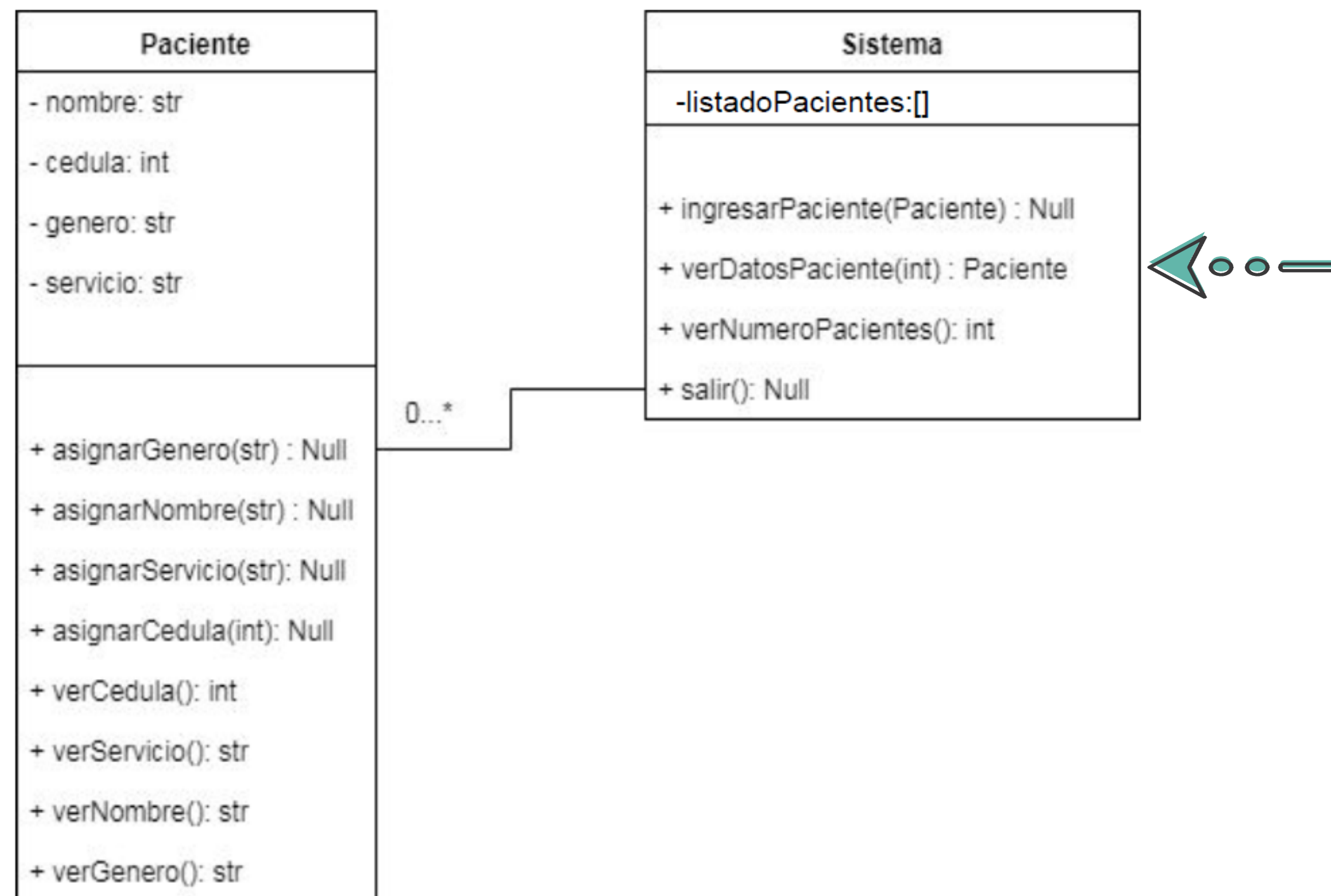
## Segundo ejercicio de abstracción, Versión 2

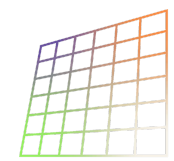
1. **Ingresar un paciente Nuevo:** Se solicita la información desde el método main y se pasa un objeto Paciente a la clase Sistema
2. **Ver todos los datos de un paciente existente:** Se pasa el número de cédula y la clase Sistema devuelve el objeto asociado a esa cédula
3. **Ver número de pacientes en el sistema:** Muestra la cantidad de personas ingresadas
4. **Salir:** Entrega un mensaje de despedida y termina el programa

**Lógica de funcionamiento versión 2:** Usando la **función main**. La idea es que haya un método principal desde donde se gestiona la entrada de datos por teclado y la salida de información a Pantalla



# Diagrama UML, 2do ejercicio Versión 2



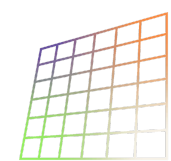


## Lógica del programa, Versión 3

La clase persona no varía , pero la clase sistema si. Para el manejo de información se añade un método que verifique en la lista cuando una cédula ya ha sido ingresada.

```
class Sistema(object):  
    def __init__(self):  
        self.__lista_pacientes = [];  
    def verificarPaciente(self, cedula):  
        #variable bandera  
        encontrado = False;  
        #voy a buscar paciente por pacietne  
        for p in self.__lista_pacientes:  
            #por cada paciente de la lista, le digo al paciente que me  
            #retorne la cedula y la comparo con la ingresada por teclado  
            if cedula == p.verCedula():  
                encontrado = True; #si encuentro el paciente actualizo la bandera  
                break; #salgo del para  
        return encontrado;
```





## Lógica del programa, Versión 3

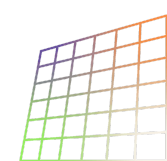
De este modo es posible verificar si un paciente existe en la BD (Lista) antes de ser ingresado al sistema

**NOTA:** NO es la única forma de realizar dicha verificación, esta es solo una de tantas alternativas que cada uno podría idearse



```
def ingresarPaciente(self, pac):  
    #verifico primero si existe:  
    if self.verificarPaciente(pac.verCedula()):  
        return False;  
    #si no existe lo agrego y retorno verdadero  
    self.__lista_pacientes.append(pac);  
    return True;
```





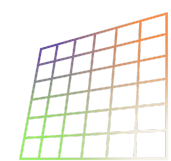
## Lógica del programa, Versión 3

Notese que creamos un método de verificación dentro de la clase *Sistema()* , que retornara una valor booleano como lo mostramos en el diagrama UML, variando la lógica de del *main()*, y se podra dar confirmación al usuario de dicho proceso.

```
if opcion == 1:
    #ingreso pacientes
    print("A continuacion se solicitaran los datos ...");
    #1. Se solicitan los datos
    nombre = input("Ingrese el nombre: ");
    cedula = int(input("Ingrese la cedula: "));
    genero = input("Ingrese el genero: ");
    servicio = input("Ingrese el servicio: ");
    #2. se crea un objeto Paciente
    pac = Paciente();
    #como el paciente esta vacio debo ingresarle la informacion
    pac.asignarCedula(cedula);
    pac.asignarGenero(genero);
    pac.asignarNombre(nombre);
    pac.asignarServicio(servicio);
    #3. se almacena en la lista que esta dentro de la clase sistema
    resultado = sis.ingresarPaciente(pac);

    if resultado == False:
        print("Ya existe un paciente con esa cedula");
    else:
        print("Paciente ingresado");
```





## Lógica del programa, Versión 3

Podremos verificar igualmente al momento de solicitar la información del paciente

```
def verDatosPaciente(self, c):  
    #verifico primero si existe, si no existe devuelvo None  
    if self.verificarPaciente(c) == False:  
        return None  
    #voy a buscar paciente por pacietne  
    for p in self.__lista_pacientes:  
        #por cada paciente de la lista, le digo al paciente que me  
        #retorne la cedula y la comparo con la ingresada por teclado  
        if c == p.verCedula():  
            return p;#si encuentro el paciente lo retorno
```





## Problemas adicionales para profundizar..



### Proponer mejoras al sistema desarrollado

**Ver todos los datos de un paciente existente:** Se pasa el número de cédula o el nombre del mismo, puede ser completo o como se recuerde, y la clase Sistema devuelve el **objeto** asociado a esa cédula o los pacientes que inician con dicho nombre

GRACIAS

